Big Data Analytics of Amazon Electronics Reviews

[M1] LIU Jiaao
25469568@life.hkbu.edu.hk
[M2] TIAN Qinan
25456431@life.hkbu.edu.hk
[M3] CHEN Yuqiao
25469746@life.hkbu.edu.hk
[M4] WANG Kuiyan
25422006@life.hkbu.edu.hk
[M5] XIE Youxin
25450875@life.hkbu.edu.hk

ABSTRACT

The goal of this project is to appraise e - commerce reviews and price fluctuations of high - end digital and gaming products with big data technologies. The study will amass a significant amount of data from digital platforms like Amazon and Kaggle, including reviews of products, ratings, and historical prices. PySpark will be used to handle the data, which will then be stored in distributed databases such as HDFS and MongoDB. In addition to that, sentiment analysis and trend analysis will be conducted by means of Spark MLlib. Well - structured charts are used to present the results. The project shows that big data can be used to support data - driven decision - making.

1 Introduction & Objectives

1.1 Motivation

The extensive volume of large-scale e-commerce data offers unique possibilities for analyzing potential channels for product placement, pricing, and gauging consumer attitudes.

Nevertheless, the task of deriving meaningful insight from raw, semi-structured datasets like Amazon's JSONL remains especially challenging because of the high volume and high levels of noise, coupled with the complexity of combining user metadata reviews. In order to overcome the data engineering challenges, the proposed project will design a custom data-structure, distributed, big-data pipeline built out of HDFS, Apache Spark, and a NoSQL structure.

The purpose of the data cleansing, alignment, and clustering of data by attributes will be to provide actionable business insights related to the ability to quantify brand premiums and sentiment-price thresholds.

1.2 Project Lifecycle & Pipeline Phases and Justification

To ensure that we are coming to our big data architecture in a systematic way, the project lifecycle is organized into four consecutive steps, right after the initial proposal with some practical improvements. This project has all four steps of big data management with a rigid inclusion of these steps. This gradual approach will guarantee a separation of concerns along the data processing pipeline:

Phase 1 Data Collection: To process the huge volume and semi-structured format of the raw Amazon JSONL records, raw data (metadata, reviews) will be required, including a NoSQL staging environment.

Phase 2 Data Preprocessing: The phase is important as the raw records have a lot of noise. It is also required to filter, cast schemas and align the raw JSONL records prior to storing them in the HDFS Parquet data lake in order to cut down drastically on the data volume and to guarantee data quality.

Phase 3 Storage and Processing & Cleansing of Data: Processing Apache Spark blocks of data and HDFS blocks of data is justified by the necessity to use distributed computing. This stage includes intensive data quality check, deleting outliers, and deduplication necessary to avoid distorted results in further machine learning models.

Phase 4 Data Analytics and Visualization: This last phase is required to convert the processed blocks of the data into actionable business knowledge. The use of machine learning clustering, business metrics (e.g., brand premium, sentiment) and visualization directly responds to our research objectives.

1.3 Team Roles and Pipeline Responsibilities

Member 1 (LIU Jiaao): P**roject Management & System Architecture** (Phase 1 & 2): Designed the containerized architecture and managed the codebase. Executed raw JSONL data ingestion via MongoDB, aligned metadata with reviews, and migrated structured data to the HDFS Parquet data lake.

Member 2 (TIAN Qinan): **Data Quality & Cleansing** (Phase 3): Engineered the distributed cleaning pipeline. Handled missing values, eliminated price outliers via the

IQR method, and deduplicated review records to build the production-ready clean layer.

Member 3 (CHEN Yuqiao): **Clustering & Business Analytics** (Phase 3 & 4): Developed multi-dimensional features (Textual, Structural, Numerical) and applied K-Means clustering to stratify product configurations. Calculated advanced metrics including brand premium and equity scores.

Member 4 (WANG Kuiyan): **Machine Learning Engineering** (Phase 4): Architected a Spark ML pipeline (TF-IDF & Logistic Regression) for sentiment classification. Mined deep insights regarding store-level satisfaction and pricing inflection points.

Member 5 (XIE Youxin): **Visualization & Dashboarding** (Phase 4): Implemented the stratified visualization strategy. Processed upstream analytical CSVs to generate reproducible, multi-dimensional charts for final business reporting.

To provide an intuitive overview of these integrated phases and individual roles, **Figure 1-1** illustrates our complete big data pipeline.

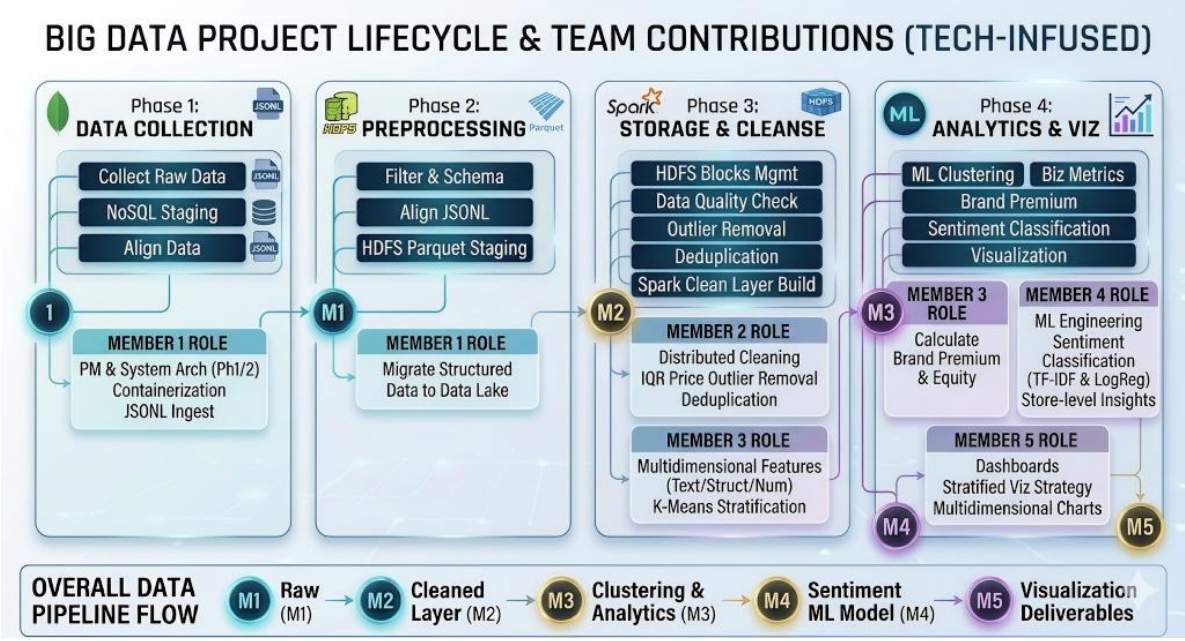


Figure 1-1: Our Big Data Project Lifecycle and Team Contributions

2 Methodology

2.1 Data Acquisition & Storage Architecture

· **Data Source:**

We used the full dataset of Amazon Reviews '23 dataset[1] offered by the McAuley Lab to find the most high-quality and relevant data to high-end digital products, concentrating on two areas: *Electronics*, *Cell Phones and Accessories*.

The raw data were of large scale as they contained more than 64 million reviews (approximately 29.7GB in the . jsonl format) and approximately 2.9 million product metadata records. We implemented a systematic filtering strategy in order to maximize the signal-to-noise ratio and guarantee computational feasibility. We selected the metadata with the help of a predefined whitelist of major technology and gaming brands (e.g., Apple, Samsung, ASUS, Logitech). After that, we used "parent_asin" field to cross reference and obtain only reviews of these verified products. The resulting dataset was eventually reduced to a high-quality set of 145, 000 products and 6.94 million reviews.

It is unexpected to note that we had an approximation of 70% missing in the "price" field. Whereas we would have accepted this sparseness as a means of saving data volume (waiting to explicitly drop or imputate at the later stages of the analysis), extreme Out-of-Memory (OOM) limitations in the context of distributed processing compelled us to change. To guarantee the stability of the pipeline, we took the lead by preemptively dropping the null "price" records.

**Detailed data audit trail is available in Appendix A.*

· **System Architecture:**

We use a fully containerized architecture, that was installed using Docker, to ensure scalability and reproducibility of the environment. The main pipeline consists of Apache Spark to support distributed ETL operations, MongoDB to support initial ingestion of large volumes of unstructured text, and a Data Lake architecture.

In the initial stages of preprocessing, we faced serious memory constraints when we were running heavy transformations on HDFS.

So finally we opted for MongoDB to manage our data ingestion. As a way of reducing this and maximizing system stability, we implemented a hybrid approach to storage: the processed and structured metadata were temporarily stored in a localized Data Lake. This architecture workaround circumvented the HDFS memory limitations without compromising the PySpark pipeline.

· **Storage Optimization:**

An important engineering choice that occurred in our pipeline was the switch to the Apache Parquet columnar storage format rather than the bloated, row-based JSONL format. We obtained an extreme data compression by storing the cleaned metadata and mapped reviews into Parquet files.

The columnar nature of parquet has a natural advantage in distributed processing with Spark; predicate pushdown and column pruning can be enabled (i.e. only columns required are read with Spark such as only reading by filtering on the shop-only "store" or "parent_asin"). This dramatically lowered disk I/O and memory usage, which formed a highly optimized base on future analytics. **Figure 2-1** illustrates the overall system architecture of our project.

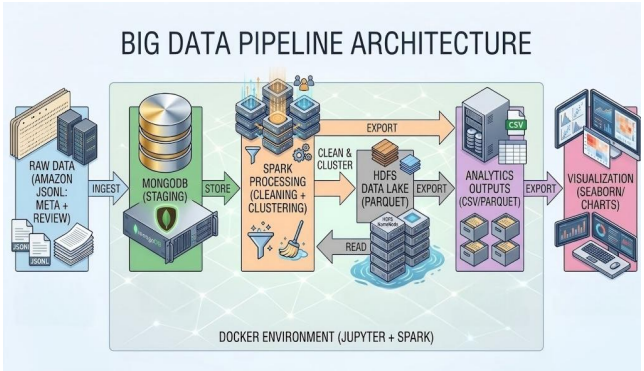


Figure 2-1: Our Project's Big Data Pipeline Architecture

2.2 Data Preprocessing & Anomaly Detection

2.2.1 ETL Pipeline

Before the data entered the analysis stage, we built a complete ETL pipeline, which was responsible for cleaning and converting the original metadata and user review data. The main processing process is shown in the following steps:

Missing value processing

Meta table: filter out records where parent_asin is empty or price is empty. Since price is a key field for subsequent price trend analysis, missing or invalid price records will be removed directly.

Review table: Filter records with empty scores, empty comment text, or missing asin/timestamps. At the same time, make sure that the score range is between 1–5.

Repeat the record to remove the weight

The de-weighting rule is based on the combination of three fields reviewerID, asin and unixReviewTime to ensure that only one comment from the same user on the same product at the same time is kept. This design can effectively prevent repeated comments generated by crawlers or data import from affecting subsequent analysis.

Unified data format

Convert the price field into numerical types and remove currency symbols and illegal characters. Unify the comment time field and generate review_date and year_month partition fields, which is convenient for subsequent analysis by time dimension.

Date storage

The cleaned data is written to HDFS in Parquet format and stored by year_month partition to improve
query efficiency.

Anomaly Detection

To identify outliers in prices, we employed a box plot method based on interquartile range.

Boxplot Principle

The box plot illustrates price distribution through the quartiles of data.Define as follows:

The price of the next quartile Q1: 25% is lower than this value

The price of the upper quartile Q3: 75% is lower than the value

Quartile distance IQR = Q3 − Q1

Criteria for outlier identification

Normal price range : [Q1 − 1.5 × IQR, Q3 + 1.5 × IQR]. The price below the lower bound is considered to be an abnormal value too low.The price higher than the upper bound is considered to be an abnormal value that is too high.

Processing process

First, filter out the records with the price of null or ≤ 0. Calculate the price of Q1, Q3 and IQR after cleaning. Eliminate extreme price records beyond the normal range.

Effect Description

This method does not rely on the data distribution hypothesis and is applicable to the price data of bias distribution. At the same time, it effectively eliminates obviously unreasonable prices and retains the main form of price distribution, which is conducive to subsequent price trend analysis. The cleaning result can be seen in **Figure 2-2**.

**IQR Algorithm is available in Appendix B*

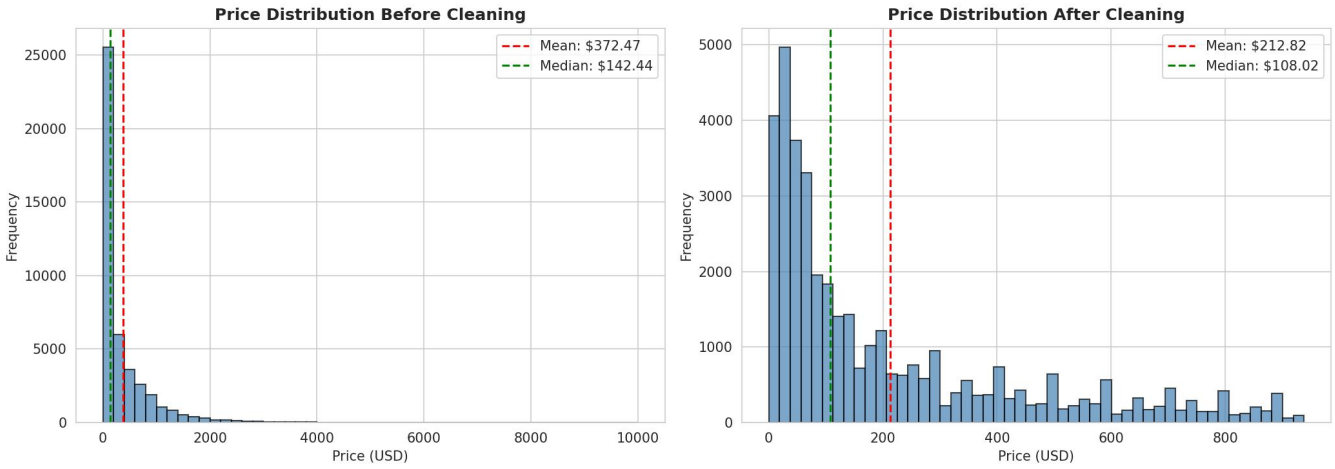


Figure 2-2: Pre-and post-cleaning comparison

2.3 K-Means Clustering for Configuration

Challenge

As far as traditional e-commerce price analysis is concerned, the biggest logical fallacy is “comparing prices without considering configurations.” For instance, comparing the average price of Apple and Redmi smartphones is not about brand premium and brand value; it’s about brand positioning. Many people assume that Apple’s brand premium is justified because it is an expensive phone as compared to Redmi, without considering that the only thing that can constitute a premium is the brand is the price assuming equivalent hardware. However, in the Amazon dataset, the details field is very unstructured. For example, a category can contain an expensive flagship smartphone, and also a very cheap phone case. This makes it very difficult to create a fair baseline for the purpose of comparison.

Solution

To construct an industry baseline for price comparison that is as fair as possible, we undertook the following deep feature engineering and unsupervised learning processing steps:

Complex Details Extraction & Hard Filtering:

We performed an extensive analysis of the unstructured details dictionaries using CountVectorizer for the configuration key-values and integrated it with Word2Vec for textual semantic feature extraction. More crucially, we engaged in conscious, targeted clustering. Instead of clustering blindly, we executed reverse feedback optimization using an empirical noise vocabulary (case, cable, etc.) identified through Exploratory Data Analysis (EDA) after we undertook our initial clustering. In this new preprocessing iteration, we applied Hard Filtering so that only 'core devices' and not, 'peripheral accessories', were introduced to the algorithm, thus effectively diminishing the intra-cluster variance.

Note, the refined forced extraction logic utilizing regular expressions for the most typical numerical attributes of electronic products, such as ram volume and screen inches, to expand the dimesion of our NLP text feature was also

one of our exploratory directions and it is in place already, which significantly reduced the confusion caused by simple semantic feature clustering.

Double-Blind K-Means Categorization:

We used a 'double blind' strategy during K-Means training inspired by clinical trials:

Blinding Brands:

We inserted prominent brand names in a custom stop word list (Custom Brands Stop Words). This compelled the algorithm to classify products based exclusively on their 'hardware attributes,' thus evading the feature co-occurrence trap.

Blinding Prices:

We completely refrained from using price as one of the clustering attributes, thus eliminating target leakage. This ensured that price was the only independent variable for subsequent premium analysis.

Establishing Fair Baselines:

The K-Means algorithm performed an objective and independent analysis of an unprecedented amount of products and divided them into different clusters in a multidimensional vector space.

With regards to business application interpretation, these clusters were related to a some local adaptation of configurations tiers mappings that match real-world markets: Entry-level High-End or even Luxury/Professional levels for special electronic categories with ceilings like computers or personal workstations.

Because of these purified "configuration groups", however, we were able to accurately compute the overall "industry baseline price" by simply performing SQL aggregates within each cluster. As a result, the quantification of the relative brand premium and real brand value became way more credible.

**KMEANS Algorithm is available in Appendix B*

3 Advanced Analytics & Model Training

3.1 Distributed SQL & Configuration Premium

Analytical Logic:

Distributed Aggregation and Confounder Control via Spark SQL. For this analysis, the distributed capabilities of PySpark SQL and the DataFrame API were used, and as a contribution to the analytical foundation, the technique of “controlling for confounders” was imported.

Creating a Single Source of Truth (SSOT):

To avoid diverging definitions of metrics due to redundant calculations at different sub-modules, a primary fact table (Item_Premium_Base) was built using Common Table Expressions (CTE) in Spark SQL. First, the market baseline price of each configuration was calculated by the hardware cluster. After that, this baseline was correctly attributed back to each SKU, and through the configuration hardware cost, the calculated price difference reflected the real brand premium or “true pricing power”.

Advanced SQL Techniques and Robustness Assurance:

Null-Handling and Zero-Division Prevention: In the case of working with a large volume of web-scraped datasets with irregularities, the COALESCE function was used to substitute null values with zero in the review count, and also the NULLIF function was used to control “divide by zero” errors, which, in distributed computing, can cause the whole process to fail.

Cluster Ranking and Semantic Labeling:

At the data analysis layer, Spark SQL windowing functions were used to sort each cluster in descending order of base price. Accordingly, commercial labels were dynamically assigned. As a result, numerical labels were replaced with qualitative classifications, such as Premium, High-End, and Entry Level. This enhanced the applicability of the data for business purposes.

Min-Max Scaling via Window Functions:

To facilitate a more intuitive user experience when working with data in the business intelligence reporting tools like the BCG Matrix, MIN() OVER and MAX() OVER window functions were employed. Values were all standardized regardless of the dispersion in the original data distribution, resulted in a semantic data scale of 0 to 100 that was highly usable.

Metric Definition:

The Dual-Index Heuristic Model and Sales-Proxy Weighting. In relation to the mathematical derivation of the metrics, a simple average price comparison was no longer deemed sufficient and a more rigorous approach using a heuristic proxy model was adopted.

Mitigating Aggregation Bias and the True Premium Rate Formula:

One of the major issues with traditional models is that they tend to calculate the direct average of prices of units, which makes them very vulnerable to positive skewness due to long-tail items that have significantly high prices but

low sales. In order to correct this bias, a "sales-proxy weighting" methodology was introduced. The logic for the calculation is defined as follows:

Configuration Premium Percentage = [(Sum of (Price * Proxy Sales)) - (Sum of (Baseline Price * Proxy Sales))] / (Sum of (Baseline Price * Proxy Sales)) * 100

Dimensionality Reduction for Commercial Value:

The Dual-Index Model: Building upon this baseline, actual consumer feedback (average rating) was integrated to derive two core multiplier indices designed to inform strategic business decisions:

1. **Brand Equity Score:**

This index is normalized based on the formula: (1 + Premium Rate) * (Rating / 5.0).It is used to find brands that command high price premiums yet still receive excellent consumer reviews, suggesting a robust brand moat plus some

2. **Value for Money Score:**

This index is normalized based on the formula: (Rating / 5.0) / (1 + Premium Rate). We follow up with a quantitative analysis that identifies brands whose prices fall well below the market baseline but have also maintained a strong reputation, which can provide evidence for platform subsidy strategies or selection of high-value products.

* *Spark SQL Algorithm can be found in Appendix B*

3.2 Sentiment Analysis Machine Learning Pipeline

Spark MLlib is used to implement the sentiment analysis to process the review data in large volumes and to extract the meaningful sentiment information in the user reviews.

NLP Pipeline:

Spark MLlib parts are used to build the pipeline. The review text is then broken down into individual words using a Tokenizer. A StopWordsRemover is then used to remove noisy words in common. TF-IDF is then applied to textual data by preprocessing with HashingTF and IDF to convert textual data into numerical feature vectors which can be used in machine learning models.

Model Training:

Sentiment classification is with the help of a Logistic Regression model. Input features consist of review text and proxy labels of sentiment are review ratings. The model is trained in a Spark ML pipeline, which allows processing a large amount of data in a distributed system efficiently.

Performance Evaluation:

The trained model is used to make sentiment predictions on the data. These are further combined into organized files like price sentiment. csv and store sentiment. csv. The reported sentiment change between various price points and stores suggests that the model is useful in representing significant patterns of user feedback, which can be used to analyse downstream, including inflection points in prices.

**TF - IDF Algorithm can be seen in Appendix B*

4 Results & Dashboards

4.1 Key Finding 1: Configuration Premium Distribution

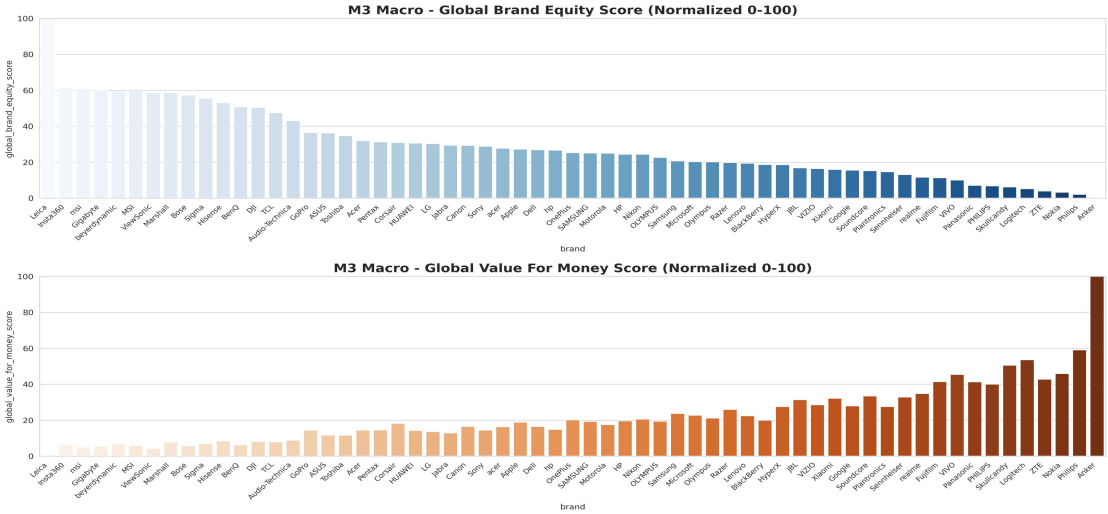


Figure 4-1: Global Brand Equity Score and Global Value For Money Score (Normalized 0 – 100) by Brands

See **Figure 4-1**, The two normalized scores, ranging from 0 to 100, present distinct stratification and positioning differences across brands. The brand equity score reflects consumers' perceived brand value and market influence, while the value-for-money score indicates the relative cost-performance ratio of each brand. By comparing these two metrics, we observe that brands with strong brand equity do not necessarily achieve high value-for-money ratings, and some mid-tier brands demonstrate more balanced performance in both dimensions. This reveals a clear trade-off between high-end brand positioning and practical cost performance, highlighting the divergent market strategies adopted by different brands in the high-end digital and gaming product industry.

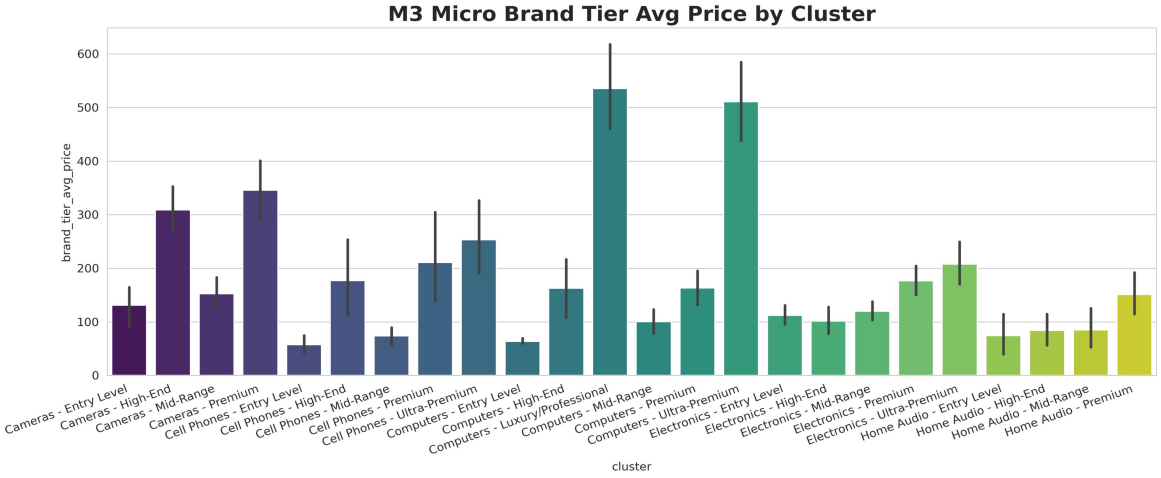


Figure 4-2: Average Brand Tier Price by Configuration Clusters

See **Figure 4-2**, Cluster-level pricing reflects the product configuration tiers. Significant price differences across clusters verify the effectiveness of K-Means clustering in distinguishing high/medium/low product configurations

4.2 Key Finding 2: Pricing Inflection Points

Sentiment Bundle by Price Buckets

The sentiment scores are produced through the Spark MLlib pipeline and aggregated in fixed price buckets of 100 price based on the `price_sentiment.csv` dataset. This compilation enables a systematic comparison of user satisfaction of various price levels.

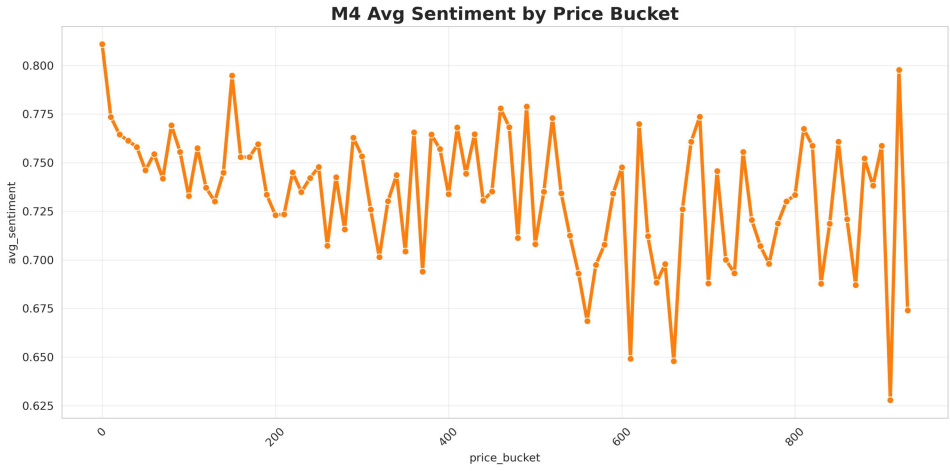


Figure 4-3: Average User Sentiment Score by $100 Price Bucket

Monitored Trend between Price and Sentiment:

The findings indicate a strong negative relationship between price increase and sentiment. The sentiment scores are comparatively high in products in the lower price range which means that the user satisfaction is high. The average sentiment decreases slowly as the prices shift towards mid-range and higher.

Identification of Pricing Inflection Point:

The inflection point is seen in the middle of the price range, with a more pronounced decrease in sentiment than in other lower price buckets. This implies that there is a certain price at which raising price does not result in a higher user satisfaction.

As visualized in **Figure 4-3**, the sentiment trend line confirms this negative correlation: user satisfaction remains stable in entry-level price buckets, then begins a gradual decline in the mid-range, and experiences a sharp drop beyond the identified inflection point. This visual validation reinforces the quantitative findings, making the pricing threshold more intuitive for business decision-making.

Store-Level Sentiment Distribution:

Beyond price-level analysis, we further examine the distribution of positive sentiment across all retail stores to evaluate overall customer satisfaction at the operational level.

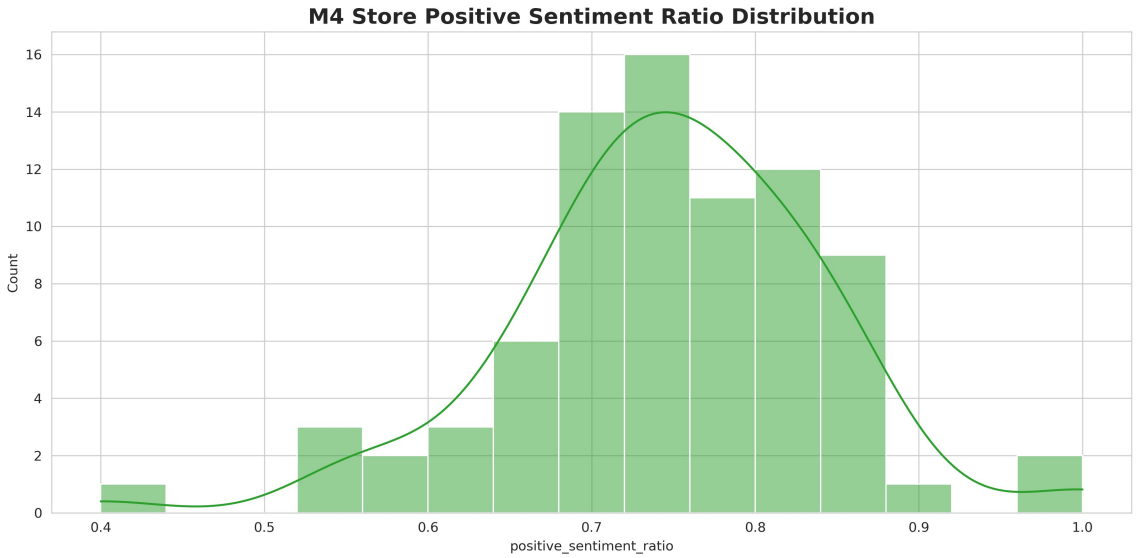


Figure 4-4: Distribution of Positive Sentiment Ratio Across Stores

The histogram in **Figure 4.4** presents a near-normal distribution of store-level positive sentiment ratios, with the majority of stores clustered around a 60–70% positive feedback rate. This distribution indicates consistent overall customer satisfaction across most locations, while a small number of outliers represent both top-performing stores with extremely high sentiment and underperforming stores requiring operational improvement. This visualization complements the price-sentiment analysis, providing a holistic view of user satisfaction from both product pricing and store operation perspectives.

4.3 Visualization Strategy

To ensure clarity, interpretability, and actionable insight delivery throughout the analysis pipeline, M5 implemented a hierarchical visualization strategy spanning macro brand analysis, micro configuration clustering, and operational sentiment monitoring.

Macro-Level Brand Stratification:

For the macro perspective, individual brand performance is quantified using two normalized scores (0–100): Global Brand Equity Score and Global Value for Money Score. As demonstrated in **Figure 4-1**, we adopted a stratified sampling approach to retain only high-variance brands, ensuring that the visual hierarchy effectively communicates

the relative market positioning and cost-performance ratio of each brand. This dual-score visualization facilitates the identification of strategic divergences between premium and mid-tier brands.

Micro-Level Configuration Clustering:

To validate the results of K-Means clustering, we visualized the Average Brand Tier Price by Configuration Cluster (**Figure 4-2**). This bar chart serves as a critical sanity check, confirming that the clustering algorithm successfully segregated products into distinct high/medium/low configuration tiers based on price differentials. The clear separation across clusters verifies the robustness of our feature engineering and clustering efficacy.

Holistic Sentiment Trend Analysis:

For the price-sentiment relationship analysis (**Figure 4.3**), a line plot was selected to intuitively illustrate the negative correlation across $100 price buckets. This visualization format was chosen for its ability to clearly highlight the pricing inflection point where user satisfaction decays. Additionally, the distribution of positive sentiment ratios (**Figure 4.4**) was visualized using a histogram to provide a comprehensive view of store-level performance, offering insights into operational consistency.

Reproducibility and Standardization:

All visualizations generated in M5 are deterministic and reproducible. The pipeline imports cleaned CSV datasets (e.g., Macro_brand_premium.csv, price_sentiment.csv) and applies standardized plotting functions (bar charts for categorical comparison, line plots for trend analysis, histograms for distribution). This standardized approach ensures consistency across figures and simplifies the debugging process for future model iterations.

5 Conclusion

This project developed a complete big data analysis pipeline for high-end digital and gaming products, covering data ingestion, preprocessing, distributed storage, clustering, sentiment analysis, and pricing insight mining. Using Spark, HDFS, and MongoDB, we built a scalable and reproducible system to process large-scale e-commerce data.

Through K-Means clustering, we effectively divided products into different configuration tiers, enabling reasonable brand premium calculation. Sentiment analysis further revealed the negative correlation between price levels and user satisfaction, as well as clear pricing inflection points that affect consumer perception. These findings help identify market positioning patterns and provide data support for pricing and operational strategies. In the future, the system can be extended to real-time data streaming and broader product categories to enhance the timeliness and applicability of analytical results.

Reference

[1]Amazon Reviews ‘23 Dataset: https://amazon-reviews-2023.github.io/

[2]García S, Ramírez-Gallego S, Luengo J, et al. Big data preprocessing: methods and prospects[J]. Big data analytics, 2016, 1(1): 9.

[3]Ogunsola K O, Balogun E D, Ogunmokun A S. Developing an automated ETL pipeline model for enhanced data quality and governance in analytics[J]. International Journal of Multidisciplinary Research and Growth Evaluation, 2022, 3(1): 791-796.

Appendix A: Data Pipeline Audit Trail

Phase	Dataset	Operation / Filter Step	In Rows	Out Rows	Loss Rows	Retention
Phase 1&2	META (elec)	JSONL → Mongo	1,610,012	1,610,012	0	100.00%
	REV (elec)	JSONL → Mongo	43,886,944	43,886,944	0	100.00%
	META (cellp)	JSONL → Mongo	1,288,490	1,288,490	0	100.00%
	REV (cellp)	JSONL → Mongo	20,812,945	20,812,945	0	100.00%
	META (elec)	Mongo → HDFS (**Brand whitelist Filter**)	1,610,012	128,428	1,481,584	8.00%
	REV (elec)	Mongo → HDFS (**Align parent_asin**)	43,886,944	5,984,931	37,902,013	13.60%
	META (cellp)	Mongo → HDFS (**Brand whitelist Filter**)	1,288,490	17,801	1,270,689	1.40%
	REV (cellp)	Mongo → HDFS (**Align parent_asin**)	20,812,945	960,550	19,852,395	4.60%
Phase 3 *(META)*	META	Filter null parent_asin	146,229	146,229	0	100.00%
	META	Filter **null/invalid** price	146,229	43,807	102,422	29.96%
	META	Price **outlier** (IQR)	43,807	39,003	4,804	89.03%
	META	Deduplicate (parent_asin)	39,003	39,003	0	100.00%
Phase 3 *(REVIEW)*	REVIEW	Filter rating range, null asin/unixTime	6,945,481	6,945,481	0	100.00%
	REVIEW	Filter **empty review** text	6,945,481	6,941,179	4,302	99.94%
	REVIEW	Deduplicate (reviewerID, asin, time)	6,941,179	6,866,305	74,874	98.92%
Phase 3&4	META	Clean → Clustered Meta	39,003	33,494	5,509	85.88%
	REVIEW	Clean → Clustered Review (Inner Join)	6,866,305	6,866,305	0	100.00%

Appendix B: Pseudocode

Algorithm 1: IQR-based Outlier Detection and Cleansing

```
Input:
  D           : dataset (table of records)
  col         : target numeric field (e.g., price)
  k           : IQR multiplier (default 1.5)
  min_valid   : minimum valid value (optional, e.g., > 0)
  approx_eps  : quantile approximation error (optional, for big data)

Output:
  D_clean     : cleaned dataset with outliers removed
  stats       : {Q1, Q3, IQR, lower_bound, upper_bound, removed_invalid, removed_outliers}

Procedure:
1. removed_invalid  <- 0
2. removed_outliers <- 0

3. // Step A: Basic validity filtering (null / non-positive / non-numeric)
4. D1 <- empty dataset
5. for each record r in D do
6.     v <- r[col]
7.     if v is NULL then
8.         removed_invalid <- removed_invalid + 1
9.         continue
10.    if min_valid is set AND v <= min_valid then
11.        removed_invalid <- removed_invalid + 1
12.        continue
13.    append r to D1
14. end for

15. if size(D1) == 0 then
16.     return D1, stats with empty bounds

17. // Step B: Compute quartiles (exact or approximate)
18. Q1 <- Quantile(D1[col], 0.25, approx_eps)
19. Q3 <- Quantile(D1[col], 0.75, approx_eps)
20. IQR <- Q3 - Q1

21. // Step C: Determine outlier bounds
22. lower_bound <- Q1 - k * IQR
23. upper_bound <- Q3 + k * IQR

24. // Step D: Remove outliers outside the bounds
25. D_clean <- empty dataset
26. for each record r in D1 do
27.     v <- r[col]
28.     if v < lower_bound OR v > upper_bound then
29.         removed_outliers <- removed_outliers + 1
30.         continue
31.     append r to D_clean
32. end for

33. stats <- {Q1, Q3, IQR, lower_bound, upper_bound, removed_invalid, removed_outliers}
34. return D_clean, stats
```

Algorithm 2: Stratification via K-Means Clustering (Category-aware)

```
Input:
  M                : cleaned metadata table
  category_col     : category field (e.g., main_category)
  id_col           : product key (e.g., parent_asin)
  title_col        : title text field
  details_col      : details/specifications field (dict/json/string)
  K_map            : mapping {category -> K} (number of clusters per category)
  Abbv_map         : mapping {category -> abbreviation} (e.g., "Computers" -> "Co")
  seed             : random seed (default 42)

Output:
  M_clustered      : metadata table with stratification label "cluster"
                     where cluster = "{Abbv_map[cat]}_{cluster_id}"

Procedure:
1. M_clustered <- empty table

2. for each category c in keys(Abbv_map) do
3.     Mc <- filter M where category_col == c
4.     if Mc is empty then continue

5.     // Step A: Feature extraction (configuration signals)
6.     // A1. Structural spec keys from details
7.     keys <- ExtractKeys(details_col)              // e.g., map_keys(details)
8.     keys <- RemoveNullOrEmpty(keys)
9.     Mc <- filter Mc where size(keys) > 0

10.    // A2. Text semantics from product title
11.    title_tokens <- Tokenize(Lowercase(title_col))
12.    title_tokens <- RemoveStopWords(title_tokens)
13.    Mc <- filter Mc where size(title_tokens) > 0

14.    // A3. Numeric specs (optional): extract capacity/size from details
15.    num_specs <- ExtractNumericSpecs(details_col)  // e.g., GB/TB, inches
16.    num_specs <- FillMissing(num_specs, 0.0)
17.    num_specs <- MinMaxScale(num_specs)

18.    // Step B: Vectorize + fuse multi-modal features
19.    v_title   <- Word2Vec(title_tokens)            // dense semantic vector
20.    v_details <- CountVectorize(keys, binary=true) // sparse config-key vector
21.    v_num     <- num_specs_as_vector(num_specs)

22.    v_all <- Concat(v_title, v_details, v_num)
23.    v_all <- L2Normalize(v_all)

24.    // Step C: K-Means clustering within category
25.    K <- K_map[c]
26.    model <- KMeansFit(v_all, K, seed)
27.    cluster_id <- KMeansPredict(model, v_all)      // integer in [0, K-1]

28.    // Step D: Assign stratification labels
29.    abbv <- Abbv_map[c]
30.    cluster_label <- ConcatString(abbv, "_", ToString(cluster_id))

31.    Mc_out <- Mc with column cluster = cluster_label
32.    M_clustered <- Union(M_clustered, Mc_out)

33. end for

34. return M_clustered
```

Algorithm 3: SSOT-Based Brand Premium & Dual-Index Analytics (Spark SQL)

```
Input:
  MetaClustered: clustered metadata table
    columns: {parent_asin, store, main_category, price, average_rating, rating_number, cluster, ...}
  Params:
    MIN_SKU_PER_TIER   : minimum SKU count threshold per (store, cluster)
    MIN_REVIEWS_PER_TIER: minimum total reviews threshold per (store, cluster)
    EPS                : small constant for numerical stability (optional)

Output:
  MicroPremiumTable: per (store, cluster) premium & indices
  MacroPremiumTable: per store overall premium & indices + per-category premium summary
  ClusterTierMap   : dynamic mapping from cluster -> semantic tier label (Entry/Mid/High/Premium/...)

Procedure:

1) Data Cleansing (robustness)
   D <- MetaClustered
   D <- filter D where price IS NOT NULL and store IS NOT NULL and store != "Unknown"
   D <- replace NULL rating_number with 0 using COALESCE

2) Build SSOT Fact Table: Item_Premium_Base (CTE idea)
   // 2.1 Compute configuration baseline price per cluster (controls confounder = configuration)
   ClusterBaseline <- group D by cluster:
       baseline_price = AVG(price)

   // 2.2 Attribute baseline back to each SKU (single source of truth)
   Item_Premium_Base <- join D with ClusterBaseline on cluster
   // Item_Premium_Base columns include: {cluster, store, main_category, price, rating_number, baseline_price,
average_rating}

3) Confounder Control via Stratified Aggregation (within-cluster)
   // Avoid comparing brands across different configurations by restricting computations within the same cluster.

4) Micro-level Premium (store × cluster) with sales-proxy weighting
   For each group G = (store, cluster) in Item_Premium_Base:
     sku_count      = COUNT(*)
     total_reviews  = SUM(rating_number)
     proxy_sales    = CASE WHEN rating_number == 0 THEN 1 ELSE rating_number  (row-wise)
     numerator      = SUM(price * proxy_sales) - SUM(baseline_price * proxy_sales)
     denominator    = NULLIF(SUM(baseline_price * proxy_sales), 0)           // prevent divide-by-zero
     premium_pct    = (numerator / denominator) * 100

     avg_rating     = AVG(average_rating)

   Filter groups where sku_count >= MIN_SKU_PER_TIER AND total_reviews >= MIN_REVIEWS_PER_TIER

   MicroPremiumTable <- output columns:
     {cluster, store, sku_count, total_reviews, premium_pct, avg_rating}

5) Dual-Index Heuristic Model (commercial indices)
   For each row r in MicroPremiumTable:
     premium_rate = premium_pct / 100
     BrandEquity  = (1 + premium_rate) * (avg_rating / 5.0)
     ValueForMoney= (avg_rating / 5.0) / (1 + premium_rate)

   Append columns {BrandEquity, ValueForMoney}
```

```
6) Cluster Ranking & Semantic Labeling (window functions idea)
   // Make cluster ids interpretable by sorting within each category prefix.
   ClusterStats <- group Item_Premium_Base by cluster:
       cluster_avg_price = AVG(price)
       cat_prefix        = prefix(cluster)   // e.g., "Co_3" -> "Co"

   For each cat_prefix:
     rank clusters by cluster_avg_price ascending using ROW_NUMBER() OVER (PARTITION BY cat_prefix ORDER BY
cluster_avg_price)
     assign semantic tier label by rank:
       1->Entry Level, 2->Mid-Range, 3->High-End, 4->Premium, 5->Ultra-Premium, ...

   ClusterTierMap <- {cluster -> semantic_label}

   // Join semantic label back if needed for reporting
   MicroPremiumTable <- join MicroPremiumTable with ClusterTierMap on cluster

7) Macro-level Aggregation (store-level overview)
   For each store s:
     compute weighted premium across all clusters using the same sales-proxy weighting principle
     compute store_avg_rating = AVG(average_rating) (or weighted by rating_number)

   MacroPremiumTable <- output columns:
     {store, overall_premium_pct, store_avg_rating, BrandEquity, ValueForMoney}

8) Min-Max Scaling for BI friendliness (window functions idea)
   // Standardize indices to [0, 100] for dashboards like BCG matrix.
   For each metric m in {BrandEquity, ValueForMoney}:
     m_scaled = (m - MIN(m) OVER ()) / NULLIF(MAX(m) OVER () - MIN(m) OVER (), 0) * 100

   Append {global_brand_equity_score, global_value_for_money_score}

9) Return MicroPremiumTable, MacroPremiumTable, ClusterTierMap
```

Algorithm 4: Sentiment Classification via TF-IDF & Logistic Regression

```
Input:
  R               : review dataset
  text_col        : review text field (e.g., text or reviewText)
  rating_col      : rating field (optional, used to build labels)
  label_rule      : function to map rating -> label (e.g., label=1 if rating>=4 else 0)
  num_features    : hashing dimension for TF (e.g., 10000)
  train_ratio     : training split ratio (e.g., 0.8)
  seed            : random seed (default 42)
  lr_max_iter     : LR max iterations (e.g., 10)

Output:
  model           : trained TF-IDF + Logistic Regression model
  P               : predictions dataset with columns {probability, prediction, label}

Procedure:
1. // Step A: Data preparation
2. R1 <- filter R where text_col is not NULL and length(trim(text_col)) > 0

3. if label is not in R1 then
4.     R1 <- add column label = label_rule(R1[rating_col])

5. // Step B: Split data
6. (Train, Test) <- RandomSplit(R1, [train_ratio, 1-train_ratio], seed)

7. // Step C: Text preprocessing
8. tokens <- Tokenize(Train[text_col])                    // words array
```

```
9. tokens <- RemoveStopWords(tokens)                        // filtered_words

10. // Step D: TF-IDF feature construction
11. tf <- HashingTF(tokens, numFeatures = num_features)     // raw term-frequency vector
12. idf_model <- FitIDF(tf)                                 // learn IDF weights from Train
13. features <- TransformIDF(idf_model, tf)                 // TF-IDF vector

14. // Step E: Train classifier
15. lr_model <- TrainLogisticRegression(features, Train[label],
                                        maxIter = lr_max_iter)

16. // Step F: Predict on test set
17. tokens_test <- Tokenize(Test[text_col])
18. tokens_test <- RemoveStopWords(tokens_test)
19. tf_test <- HashingTF(tokens_test, numFeatures = num_features)
20. features_test <- TransformIDF(idf_model, tf_test)

21. probability <- PredictProbability(lr_model, features_test)
22. prediction  <- ArgMax(probability)                      // 0/1

23. P <- Test with columns {label, probability, prediction}
24. return (lr_model, idf_model), P
```